Implementing Transformer Model Final Report

1 Introduction

Positional encoding is a critical component of Transformer models, as these architectures lack inherent sequence order awareness. While early approaches relied on fixed absolute or sinusoidal encodings [[1]], recent studies suggest that such encodings may struggle to extrapolate and generalize to unseen sequence lengths [[2]–[5]]. In this study, we investigate the effectiveness of various positional encoding schemes on synthetic sequence tasks.

Sinusoidal encoding with trainable encoding—with different initialization—across synthetic sequence modeling tasks are systematically compared. Our goal is to evaluate how these encoding methods impact both convergence speed and generalization, particularly for sequence lengths not seen during training.

By analyzing the interplay between encoding strategies, sequence length, and initialization scale, we provide insights into how different positional encodings influence model performance.[1]

2 Background

2.1 The Transformer Model

The Transformer architecture [1] introduced a paradigm shift in sequence modeling by replacing recurrent mechanisms with self-attention. This change enables parallelization and improved long-range dependency modeling. The core component of the Transformer is the multi-head self-attention mechanism, which allows tokens to attend to all other tokens within a sequence. However, unlike recurrent models, Transformers do not inherently capture word order, necessitating the use of positional encodings.

2.2 Positional Encoding in Transformers

To compensate for the lack of sequential order in self-attention, positional encodings are added to token embeddings. These encodings provide each token with information about its position in the sequence, enabling the model to differentiate between otherwise identical token embeddings.

2.2.1 Sinusoidal Positional Encoding

The original Transformer model employs sinusoidal positional encodings, defined as:

$$PE_(pos, 2i) = sin(\frac{pos}{10000^{2i/d}}), PE_(pos, 2i+1) = cos(\frac{pos}{10000^{2i/d}})$$

where pos is the token position, i is the dimension index, and d is the embedding dimension. This encoding has desirable properties:

- It enables the model to generalize to unseen sequence lengths since the encodings follow a fixed mathematical pattern.
- Relative positions are preserved through linear transformations.

2.2.2 Trainable Positional Encodings

An alternative approach is to learn positional embeddings instead of using a fixed function. These embeddings are initialized and optimized during training. We consider three initialization strategies:

- **Uniform**: Initialized from a uniform distribution, typically in the range [-0.1, 0.1].
- **Normal**: Sampled from a Gaussian distribution with mean 0 and standard deviation 0.1.
- **Standard**: Sampled from a Normal Standard Distribution with mean 0 and standard deviation 1.

[1] Check full code here: `https://github.com/DareMens/transformer_project`

- **Xavier**: Weights are initialized using the Xavier/Glorot initialization [[6]], where values are sampled from a q uniform distribution with limits calculated as $\pm\sqrt{\frac{6}{d_{model}}}$.

Trainable encodings offer flexibility but add more parameters during training.

3 Implementation Details and Model Architecture

3.1 Model Architecture

The Transformer model follows the standard encoder-decoder architecture proposed in [[1]]. Below, we describe the key components in detail.

3.1.1 Multi-Head Self-Attention

The multi-head self-attention mechanism is the core component of the Transformer. It allows the model to attend to different positions of the input sequence simultaneously. For a given input sequence of embeddings $X \in \mathbb{R}^{n \times d}$, where n is the sequence length and d is the embedding dimension, the attention mechanism computes:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

where Q, K, and V are the query, key, and value matrices, respectively, obtained by linear transformations of X. The multi-head mechanism extends this by applying multiple attention heads in parallel and concatenating their outputs:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O,$$

, where each head is computed as:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V).$$

Here, W_i^Q , W_i^K , W_i^V , and W^O are learned projection matrices.

3.1.2 Layer Normalization

Layer normalization is applied after each sub-layer (attention and feedforward) to stabilize training and improve convergence. For an input x, layer normalization computes:

$$LayerNorm(x) = \gamma.\frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where μ and σ^2 are the mean and variance of x, γ and β are learnable scaling and shifting parameters, and ϵ is a small constant for numerical stability. Layer normalization ensures that the activations have zero mean and unit variance, which helps mitigate the vanishing gradient problem.

3.1.3 Position-wise Feedforward Layer

The position-wise feedforward layer consists of two linear transformations with a ReLU activation in between:

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2.$$

This layer is applied independently to each position in the sequence, allowing the model to capture complex patterns in the data.

3.1.4 Residual Connections

Residual connections are used around each sub-layer to facilitate gradient flow and enable deeper architectures. The output of each sub-layer is computed as:

$$Output = LayerNorm(x + Sublayer(x)),$$

where Sublayer represents either the multi-head attention or feedforward layer.

3.1.5 Embedding and Positional Encoding

The model uses a shared word embedding layer for both the encoder and decoder, tied with the output projection layer to reduce the number of trainable parameters. The input embeddings are augmented with one of the positional encoding methods discussed in Section 2.

To investigate the effect of different encoding strategies, we replace the standard sinusoidal encoding with:

- Trainable positional encodings (Uniform).
- Trainable positional encodings (Normal).
- Trainable positional encodings (Standard).
- Trainable positional encodings (Xavier).

3.2 Training Configuration

For training, we use the AdamW optimizer with a learning rate schedule similar to [[1]], defined as:

$$lr = d_{model}^{-0.5} \times min(step^{-0.5}, step \times warmup_steps^{-1.5})$$

where d_{model} is the hidden size, and warmup steps is set to 4000. Dropout with a probability of 0.1 is applied after each attention and feedforward sub-layer to prevent overfitting, and label smoothing is applied with a smoothing factor of 0.1 to improve generalization.

3.3 Implementation Details

Our model is implemented using PyTorch, leveraging the Hugging Face transformers library for tokenization and pretraining utilities. The key implementation choices include:

- **Tokenizer:** Byte-Pair Encoding (BPE) with a vocabulary size of 50,000, shared for both English and German.
- **Batch Size:** 128 sequences per batch.
- **Sequence Length:** 64 tokens.

3.4 Evaluation Metrics

The translation model is evaluated using BLEU score with greedy autoregressive generation. However, due to computational constraints and the limited sequence length of 64, performance on large-scale datasets like the WMT 17 German-to-English dataset, which consists of 5.9 million sentence pairs remains suboptimal. Instead, our focus shifts to analyzing positional encoding impact in smaller-scale controlled experiments. The next section presents our experimental setup and methodology.

4 Experiments

In this section, we describe the setup and execution of experiments to investigate the effect of different positional encoding methods on model performance. We explore the impact of encoding types and sequence lengths on two synthetic sequence tasks: `ODD_EVEN` and `NEXT_TOKEN`. These experiments allow us to evaluate how well different positional encoding strategies generalize across varying model sequence lengths.

4.1 Experimental Setup

The experiments are conducted using a simple base model configuration defined by the following parameters:

- The dimensionality of the model's hidden states (d_{model}): 64
- The number of attention heads in the multi-head attention mechanism (n_{head}): 2
- The number of layers in the encoder and decoder ($\#layers$): 2
- The experiments are run for 50 epochs with a learning rate of 1 x 10^{-4} and a batch size of 128.

4.2 Encoding Types

We evaluate the following positional encoding strategies:

- `sinusoidal:` Standard sinusoidal positional encoding [[1]].
- `trainable uniform:` Trainable positional encoding initialized with a uniform distribution.
- `trainable normal:` Trainable positional encoding initialized with a normal distribution.
- `trainable standard:` Trainable positional encoding initialized with a standard normal distribution.
- `trainable xavier:` Trainable positional encoding initialized using Xavier initialization.

Each encoding method is tested in combination with different model configurations and sequence lengths.

4.3 Task Types

Two synthetic sequence tasks are used in the experiments:

- `NEXT_TOKEN:` In this task, the model is asked to predict the next token in a sequence. The sequence is generated from a fixed range of values, and the task involves predicting the next token given the previous tokens in the sequence.
- `ODD_EVEN:` In this task, the model is provided with a sequence of random integers and is asked to classify each token as either odd or even. The target is a binary label, where 1 indicates the token is odd, and 0 indicates the token is even.

4.4 Dataset Generation

We generate datasets with varying sequence lengths, with the full lengths for testing defined as:

$$\text{sequence lengths} \in 64,\ 128,\ 256,\ 512,\ 1024$$

For training and validation, however, the sequence lengths are set to half of the corresponding test sequence lengths. This design choice aims to introduce an additional challenge in generalization, as models must extrapolate from shorter training sequences to longer, unseen test sequences. For the `NEXT_TOKEN` task, the sequence length is determined by randomly selecting a pivot index within the vocabulary and generating a subsequence; the target for each sequence is the next token in the sequence. For the `ODD_EVEN` task, each sequence consists of random integers ranging from 1 to the defined sequence length, and the target is a binary label indicating whether each token is odd or even.

The dataset is split into training, validation, and test sets with 8000 samples in the training set and 2000 samples each in the validation and test sets.

4.5 Model Training

The model is trained for 50 epochs using the generated datasets. The experimental setup allows us to evaluate the impact of different encoding strategies on both tasks under various conditions. The model's performance is evaluated based on accuracy, with the results collected after each epoch and the best validation accuracy recorded. The results are then summarized in terms of the number of model parameters, encoding type, task type, and sequence length.

5 Results and Discussion

5.1 Convergence Analysis: Epochs vs. Sequence Lengths

To understand the impact of encoding type on training convergence, we analyze the number of epochs required to reach 100% validation accuracy (or hit the 50-epoch limit). Note that training and validation samples consist of sequences that are half the length of the test samples, thereby adding an extra challenge to the learning process. In our experiments, we also incorporate Xavier initialization for trainable positional encodings to improve weight initialization. Additionally, we compare different initialization scales to emphasize the importance of small initial weights (e.g., a standard deviation of 0.1) versus larger standard deviations.

Figure-1 illustrates the relationship between the number of training epochs and the sequence lengths (vocabulary sizes) for different encoding strategies across the two tasks. Key observations include:

- **Faster Convergence for Certain Encodings:** Models with trainable uniform, trainable normal, and trainable xavier encodings reach 100% validation accuracy in fewer epochs compared to fixed sinusoidal and trainable standard variants. This indicates that learnable parameters, when properly initialized with small weights (using a standard deviation of 0.1), can quickly adapt to the data even with limited training samples.
- **Effect of Initialization Scale:** The trainable standard initialization, which uses a larger standard deviation, consistently shows slower convergence and poorer test accuracy. This underscores the importance of small initialization weights for achieving both stable convergence and good generalization.
- **Impact of Sequence Lengths:** As the sequence length increases, the number of epochs required for convergence tends to increase. This suggests that larger input distributions necessitate more training. However, the rate of increase varies by encoding type. Sinusoidal encoding shows a relatively stable convergence rate even for longer sequences.
- **Task-Specific Trends:** The differences in line styles (which represent the two tasks) reveal that the shift task generally converges faster than the odd-even task. This implies that the positional information required for the odd-even task is inherently more complex.

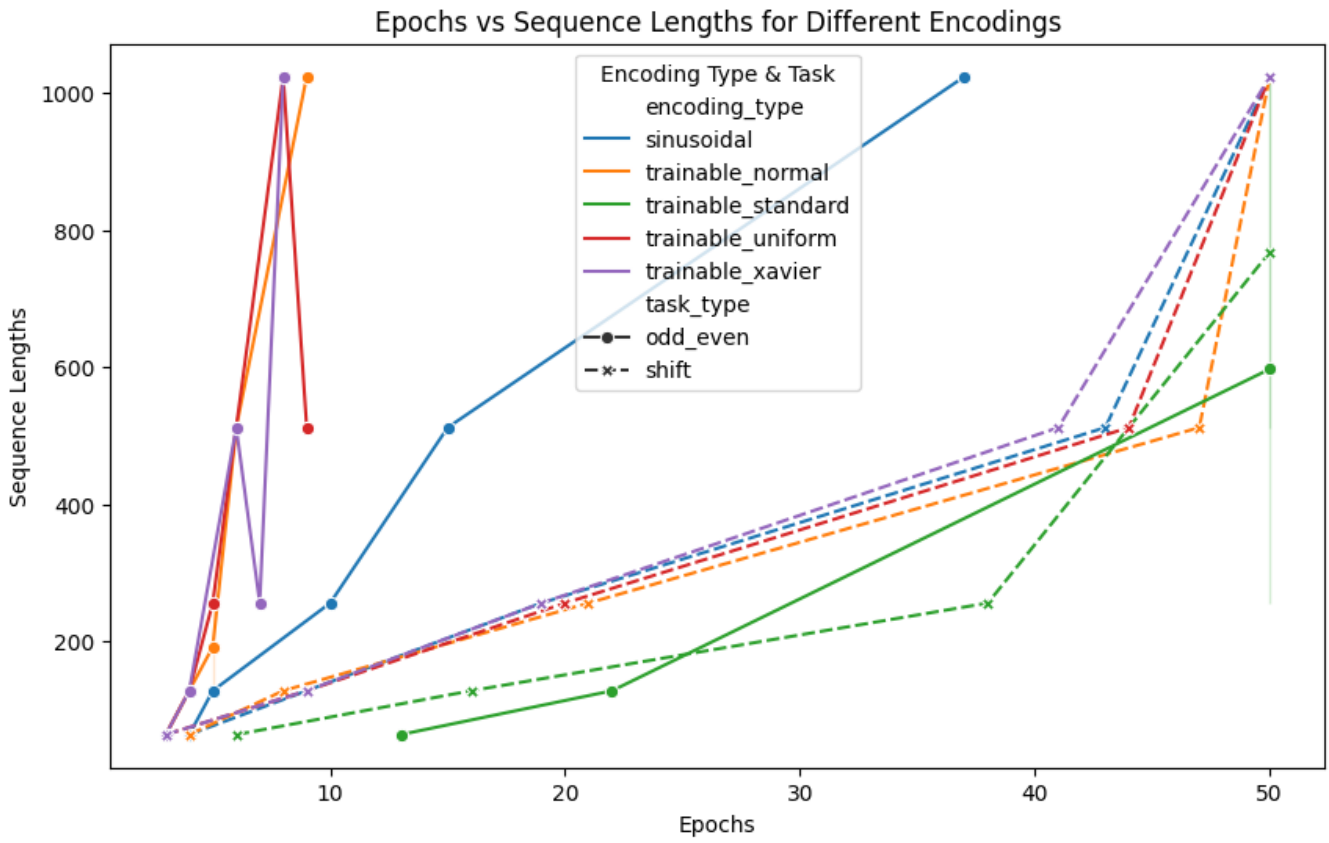

Figure 1: Epochs vs. Vocabulary Size for Different Encodings. This plot shows the number of training epochs (up to 50) required to reach 100% validation accuracy (or the epoch limit) for different sequence lengths and encoding strategies.

5.2 Generalization to Unseen Data: Test Accuracy

To evaluate generalization performance, we investigate test accuracy (measured on unseen, full-length sequences) in Figure-2. Key insights include:

- **Generalization Gap:** In many cases, test accuracy is slightly lower than validation accuracy, highlighting the inherent difficulty in generalizing to unseen sequence lengths. This gap is most pronounced for the sinusoidal encoding, where high validation accuracy does not translate into strong test performance.

- **Encoding Type Impact:** Well-initialized trainable variants show a closer alignment between validation and test accuracies, suggesting robust generalization. In contrast, sinusoidal encoding exhibits a notable drop from validation to test performance.
- **Effect of Sequence Length:** As sequence lengths increase, maintaining robust generalization becomes more challenging, particularly for encoding methods that rely on learned parameters.

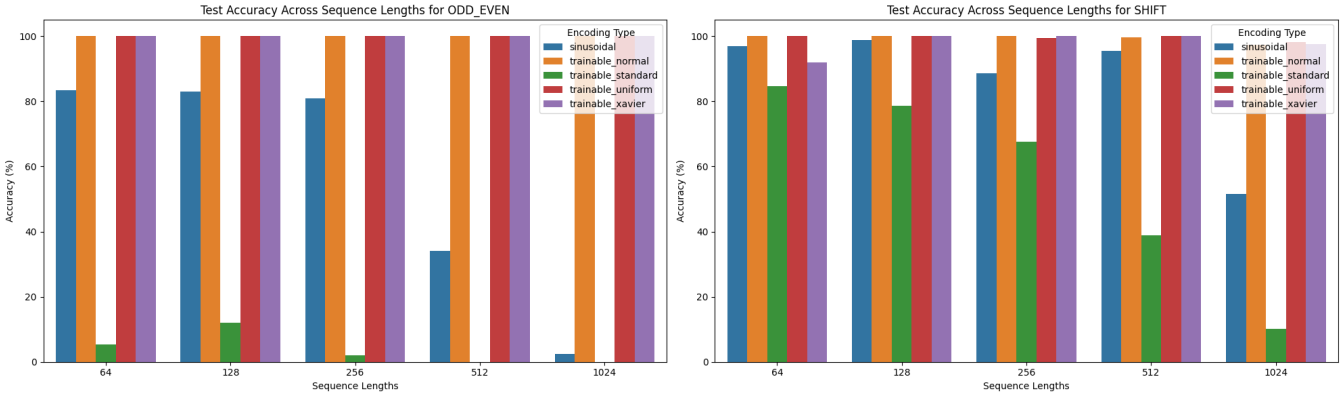

Figure 2: Test Accuracy Across Sequence Lengths. This plot illustrates the performance on unseen (test) data, highlighting the generalization capacity of each encoding strategy.

5.3 Model Complexity Growth: Sinusoidal vs. Trainable Encodings

Figure-3 examines how the number of model parameters grows as a function of sequence lengths for different encoding strategies. Our findings include:

- **Linear Growth:** Model size increases linearly with vocabulary size across all encoding types.
- **Parameter Efficiency:** Despite the increased parameter count, well-initialized trainable encodings not only converge faster but also generalize well, thereby justifying their higher parameter requirements.
- **Compactness of Sinusoidal Encoding:** The fixed sinusoidal encoding maintains a lower parameter count, which can be beneficial in scenarios with limited computational resources.

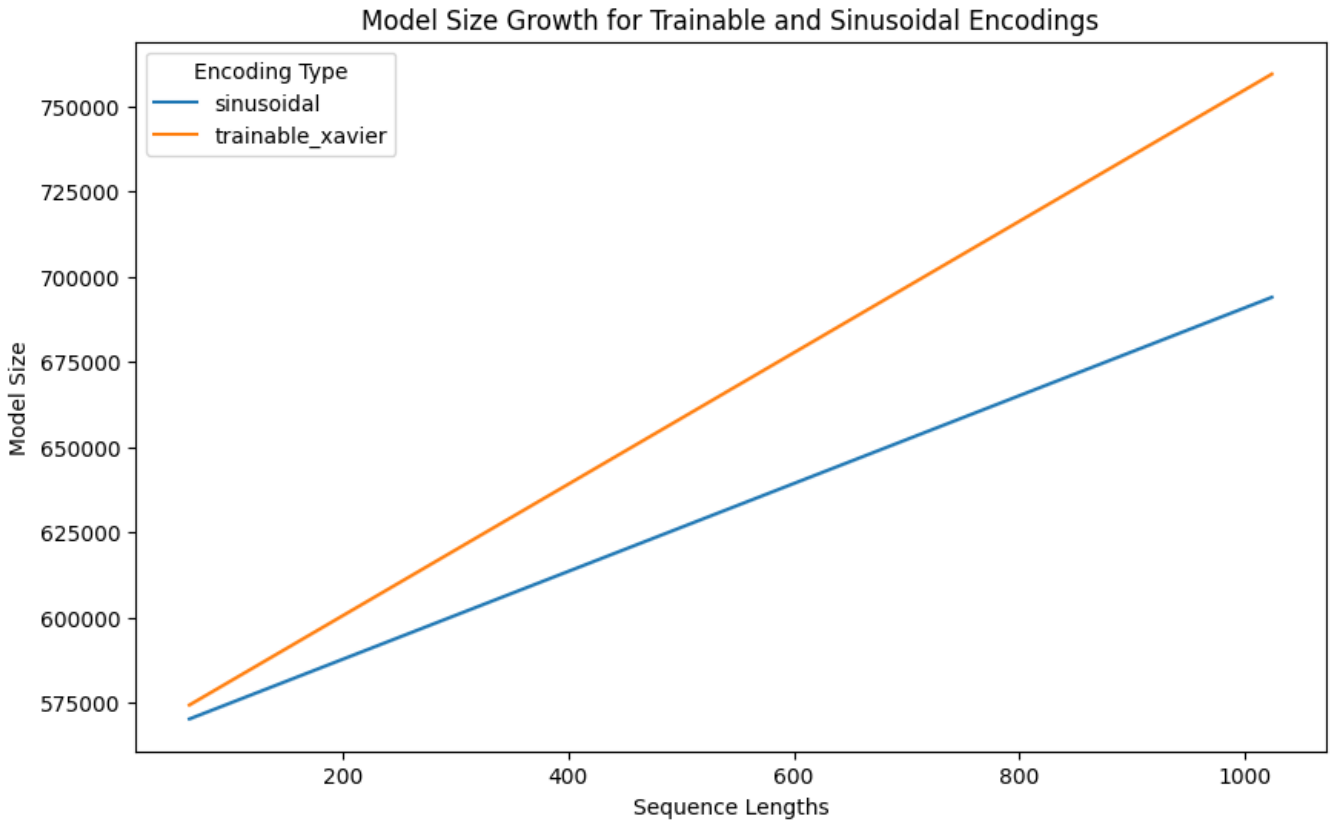

Figure 3: Model Size Growth vs. Sequence Lengths for Different Encodings. This plot illustrates the increase in model parameters with vocabulary size for both sinusoidal and trainable encodings.

5.4 Conclusion of Results

The overall findings from our experiments highlight a trade-off between convergence speed, model complexity, and generalization:

- **Convergence vs. Complexity:** While trainable encodings (especially with small, Xavier-initialized weights) converge faster, they also require a larger number of parameters compared to the sinusoidal.
- **Importance of Initialization Scale:** The results clearly show that using small initial weights (with a standard deviation of 0.1) is critical for effective convergence and generalization. The trainable standard initialization, which employs larger weights, performs significantly worse, thereby demonstrating the sensitivity of the training process to the initialization scale.
- **Generalization:** Fixed sinusoidal encodings and well-initialized trainable encodings exhibit strong generalization to unseen sequence lengths. The fixed sinusoidal encoding remains a strong candidate for tasks that demand robust generalization across varying sequence lengths without adding any overhead.
- **Model Complexity:** Trainable encodings incur a higher parameter cost as sequence length increases, potentially leading to increased computational demands. In contrast, sinusoidal encoding offers a more parameter-efficient alternative. These insights can inform the selection of positional encoding strategies for Transformer models, balancing computational constraints with desired performance outcomes.

6 Conclusion

In this work, we investigated the effectiveness of trainable and sinusoidal positional encodings in transformer models through small-scale synthetic experiments. Our findings highlight that sinusoidal encoding, while theoretically designed for extrapolation, does not always generalize effectively to unseen sequence lengths in practical settings. In contrast, trainable encodings offer greater flexibility, particularly when initialized with small weights. Future work could explore extending these experiments to larger datasets and more complex tasks to further validate the observed trends.

References

[1] aswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need. In Advances in Neural Information Processing Systems 30* (pp. 5998–6008).

[2] Press, O., Smith, N. A., & Lewis, M. (2022). Train short, test long: Attention with linear biases enables input length extrapolation. *The Tenth International Conference on Learning Representations (ICLR 2022).*

[3] Zhao, L., Feng, X., Feng, X., Zhong, W., Xu, D., Yang, Q., Liu, H., Qin, B., & Liu, T. (2024). Length extrapolation of transformers: A survey from the perspective of positional encoding. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024* (pp. 9959–9977).

[4] Ruoss, A., Delétang, G., Genewein, T., Grau-Moya, J., Csordás, R., Bennani, M., Legg, S., & Veness, J. (2023). Randomized positional encodings boost length generalization of transformers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (Volume 2: Short Papers, pp. 1889–1903)

[5] Kazemnejad, A., Padhi, I., Natesan Ramamurthy, K., Das, P., & Reddy, S. (2023). The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems* (NeurIPS 2023).

[6] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*(Vol. 9, pp. 249–256).